SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation Supplementary Material

Anonymous CVPR submission

This supplement gives the model and training details of SE(3)-WAM, including pretraining on real robot data (Appendix A), the benchmark and the evaluation protocol (Appendix B), the real-robot setup (Appendix C), additional results (Appendix D), the error decomposition that motivates the design (Appendix E), and additional related work (Appendix F). Colored dashes and bracketed entries denote values to be filled from the final runs.

A. Model and Training

A.1. Scene and Summary Tokens

Current image. A frozen MAE ViT-B/16 [18] encodes the 224×224 agent-view RGB into 196 patch tokens; we chose it over frozen DINOv3 encoders [41] and a CNN trained with the policy on development scenes (Appendix D.5). Each patch receives a view embedding, a linear embedding of its mean valid metric depth, and a linear embedding of the base-frame position of its back-projected center pixel, computed with the camera intrinsics and extrinsics. These positions also enter 3D rotary position embeddings of the patch queries and keys, and the same center points are the source points of the sparse 3D flow (Appendix A.2). Target, robot, summary, and action tokens receive no rotary embedding. Past images never enter the model, so the scene holds 212 tokens.

Target-motion history. The target history covers the last 8 control steps, at the offsets $-350, -300, \ldots, 0$ ms. Each of the 8 target-state tokens is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3, 3)$. Slots 0–6 describe the seven intervals between consecutive offsets, and slot 7 holds the current absolute pose in the base frame. Velocities divide interval displacements by the elapsed time, and accelerations divide adjacent velocity differences by the spacing of the interval centers. We standardize each dimension with training statistics, zero invalid dimensions, and append three validity flags (pose, interval, and acceleration) before a $28 \to 1024 \to 1024$ GELU MLP with an additive relative-time embedding. A slot without any valid feature is masked out of attention. On the real robot, we resample the tracker's estimates (Appendix C.2) at the same offsets, linearly in translation and along the geodesic in rotation, without extrapolation and without interpolating across gaps longer than two camera periods.

Robot state and summary tokens. 8 end-effector states (position, rotation vector, and measured gripper opening fraction) at the same offsets form the robot tokens. 16 learned embeddings form the summary tokens. The policy has no language token, and the view, time, depth, and position embeddings modify existing tokens rather than adding slots.

A.2. Forecast Heads

All forecast heads are light readouts of the final hidden states of the world stream. They are trained jointly with the action stream and skipped at deployment, and no token reads their outputs.

Target. A linear map from the mean final state of the summary tokens gives $\{(\hat{\mathbf{d}}_h, \hat{\boldsymbol{\phi}}_h)\}_{h=1}^{8}$, trained with $\mathcal{L}_P$ of Eq. 4 and $\lambda_R = 0.1$. We use the left-relative rotation vector and the cumulative translation instead of a twist exponential, which would couple translation to rotation through its Jacobian. Squared-error training gives a point estimate, which averages over multiple possible futures, e.g., before an impact.

Sparse 3D flow. The flow is the cumulative base-frame displacement of the 196 patch center points at the 8 horizons, together with their visibility. A linear readout on each patch token gives its point's displacement and visibility logit at every horizon, trained with $\mathcal{L}_F$ of Eq. 5 and the scale $s = 0.7154605$ m. The visibility loss has weight 1 in simulation, and points that are valid but invisible still supervise their displacement. The points cover the target, the robot, and the surroundings alike.

Auxiliary targets. Linear readouts of the final patch tokens predict three future appearance targets at every horizon: metric depth and moving-surface RGB through 16×16 pixel readouts per patch, and the 1024-dimensional features of a frozen DINOv3 ViT-L/16 teacher [41] on the 14×14 grid.

Depth uses squared error on valid pixels, the DINO features squared error after channel-wise layer normalization, and RGB squared error on surfaces that move by more than 1 mm, including the robot. A $1024 \to 256 \to 3072$ MLP with GELU and a final sigmoid reads the mean of the final summary tokens and reconstructs the current target's 32×32 RGB region with an absolute error. The four losses form $\mathcal{L}_{\text{aux}}$, each with weight $\lambda_{\text{aux}} = 0.1$.

A.3. Action Stream and Backbone

Commands. Each chunk has 8 seven-dimensional commands at a control period of 50 ms, each with a base-frame translation increment, a left-relative rotation vector, and a gripper command. Translation and nonzero rotation increments refer to the measured end-effector pose at application. Command j of the chunk for query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$. The task's command delay d_{task} is added on application (Eq. 1); it does not shift the training indices. Normalization uses the mean and population standard deviation of the valid issued commands of the training split. Flow matching uses a unit Gaussian in these coordinates with one flow time u per chunk, which the action tokens receive through adaptive layer normalization. Each prediction position draws 8 independent noise–time pairs and averages their losses, which reduces the variance of the action gradient without enlarging the batch. At deployment, we integrate 5 Euler steps, reverse the normalization, decode the rotation vectors, and clip each axis to 0.05 m and 0.5 rad; the gripper follows the sign of its command.

Backbone and mask. Each of the 24 layers applies one attention over all tokens with shared query, key, value, and output projections, followed by separate SiLU feed-forward networks (ratio 4) for the world and the action tokens, for 504.0M backbone parameters. The mask acts on the scene ($\mathcal{S}$), the summary tokens ($\mathcal{M}$), and the actions ($\mathcal{A}$): scene tokens attend to $\mathcal{S}$, summary tokens to $\mathcal{S}$ and $\mathcal{M}$, and action tokens to $\mathcal{M}$ and $\mathcal{A}$. A blocked edge has the logit $-\infty$ in every layer and head, in training and at deployment. Because the world tokens attend neither to the actions nor to the flow time, their states are computed once per query and reused across the Euler steps.

A.4. Training Recipe

Table S1 lists the recipe on Dynamic-14. Every control step of a demonstration is a query window, which gives 1,324,374 training windows from the 11,200 demonstrations; 350 further episodes form the validation split. Each window contains 8 query positions with their own causal histories and no attention across positions. Each loss is normalized by its count of valid labels over the accumulated global batch. The DINOv3 features of future frames are cached once per episode. We chose the depth of the model among 12, 24, and 48 layers at width 1024 on development scenes at equal numbers of training samples (Fig. S1a).

Table S1. Training hyperparameters on Dynamic-14.

Item	Value
Backbone	24 layers, width 1024, 16 heads, FFN 4096, 504.0M parameters
Tokens	212 scene, 16 summary, 8 action
Encoder	frozen MAE ViT-B/16, 224×224 input
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 10^{-4}
Learning rate	2×10^{-4}, 5,174 warm-up updates from 0, cosine decay to 0
Batch, updates	global batch 512 windows, 20 epochs (51,740 updates)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Precision	trainable weights FP32, frozen encoder BF16, gradient clipping at 0.1
Action noise	8 noise–time pairs per query position
Sampling	5 Euler steps for the action
Inference	FP32, world states cached within a query
Hardware	[N] RTX PRO 6000 GPUs, [X] hours per run

A.5. Pretraining on DROID

Data. The real-robot base trains on DROID [25], recorded at 15 Hz, together with the 3D point tracks and optimized camera calibrations that PointWorld released for it [20]. We associate each released clip with its raw DROID episode by exact agreement of the joint sequence, end-effector poses, gripper states, commands, and camera identity, and exclude clips with a non-monotonic capture clock or an ambiguous association. The model sees the first exterior camera, its depth, and PointWorld's calibration of it. Of 42,935 associated episodes, 42,087 yield usable windows: 538,740 anchor windows from 37,874 training episodes and 60,149 from 4,213 validation episodes, with no shared episode (Table S2). All times are capture timestamps. The robot history uses the 8 most recent states at or before the query time, and the flow and DINOv3 targets keep PointWorld's record times rather than being interpolated to 50 ms horizons. Each anchor expands into 8 consecutive query positions with their own causal histories; the anchor carries the flow and DINOv3 labels, and the other positions carry action labels only. Robot states and commands are expressed in the base frame after the calibrated flange-to-TCP transform. We keep the 96.88% of action slots at which the collector issued a command, without a filter on magnitude or speed, so that the base is not biased toward slow motion.

What the base learns. DROID provides no object poses, so the base has no target-state tokens and no pose head. Its

Table S2. **DROID pretraining data.** Yield of each stage and validity of the labels. Windows are causal observation windows with aligned labels, one per control step of the main camera. Flow labels count point–horizon slots.

	Train	Val.	All
Episodes associated	–	–	42,935
Episodes with windows	37,874	4,213	42,087
Windows	538,740	60,149	598,889
Valid actions (%)	—	—	96.88
Valid flow labels (%)	—	—	47.05
Future frames for DINO	—	—	3,425,218
Windows with moving objects (%)	—	—	—

Figure placeholder: Fig. S2 — model scale and pretraining

(a) Simulation model. y: development SR; x: 12, 24, and 48 layers at equal sample exposure; the selected configuration starred. **(b) DROID base.** x: training samples (log); y: validation flow endpoint error at the last horizon (left axis, mm) and action loss (right axis) for the 48-layer run; marker at the selected checkpoint.

Figure S1. Scale of the simulation model (a) and pretraining of the DROID base (b).

inputs are the current RGB-D frame and the robot history (204 scene tokens), with unavailable depth masked, and its objectives are the action loss (weight 1), the sparse 3D flow (weight 1), and the future DINOv3 features (weight 0.1). PointWorld's tracks record only visible positions, so the flow labels contain no negative visibility examples, and the visibility loss is off. Most objects in DROID move only when the robot moves them, so the base mainly learns metric geometry, the motion caused by the robot's own actions, and its coupling with the actions; the exogenous motion of our tasks and the target pose enter during fine-tuning.

Flow labels. Each patch of the 14×14 grid follows one PointWorld track over the window. Among the tracks that start inside the patch, are initially visible, and have a valid positive depth, we prefer the one with the largest image motion whose 3D displacement agrees with its image motion, and otherwise take the track nearest to the patch center. This rule covers moving objects that a fixed center point would miss: on 18 segments with visible object motion, it placed a dynamic track inside the object in 13, against 5 for the center rule. The label is the track's displacement from its initial position, and patches without a valid track are invalid; 47.05% of the point–horizon slots carry a label. On image-static surfaces, the selected tracks move by [0.9] and [5.8] mm (median and 95th percentile), against [18] and [72] mm for flow lifted from 2D correspondences [34] with monocular metric depth [45], so we use the PointWorld tracks.

Recipe. The base uses 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates, 2,106 warm-up updates) at a global batch size of 512 on 64 GPUs, with the optimizer and precision of Table S1 (Fig. S1b).

Fine-tuning. Each real-robot policy is initialized from the base and fine-tuned on the demonstrations of its task. The target-state tokens, the pose head, and the appearance heads that the base lacks start from fresh initialization, and the policy receives poses from the tracker of Appendix C.2. No real-robot policy is initialized from simulation weights, and the base sees no simulation data.

A.6. Deployment

Each query encodes the newest observation, runs the world stream once without its heads, and integrates the action stream on the cached world states. The chunk takes over at the first control-period boundary at or after the query ends, from the command meant for that boundary, and the next query starts at that boundary with a new observation. While a query runs, physics advances and the executor continues with the remainder of the previous chunk (Appendix B.3).

B. Dynamic-14 and Evaluation Protocol

B.1. Tasks

We built Dynamic-14 on MuJoCo and robosuite [43, 59]. It contains fourteen tasks in five families (Fig. S2). Table S3 lists their success conditions and Table S5 their scenario ranges. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Roles. Table S4 assigns the bodies of each task to the target and context roles of Sec. 3.1. A moving receptacle is therefore a target, since its pose decides containment. The wire of the passage tasks plays both roles: its pose at the entry is a target, because the wand must be aligned within 30 mm there, and its geometry along the loop is context, because any contact fails. Conveyors, turntables, ramps, and strikers are drivers in the context. The last column lists the error terms of Eq. 2 that each task exercises (Appendix E). The two task groups of Fig. 5 follow from this table and were fixed before the ablations: the rotation-sensitive group contains the tasks whose terms include $r\theta$ (`grasp`, `t1_so3`, `t2_se3`, `t3_phase`, `t3_full`, `t4_can`, `t5_replan`), and the context-active group contains the tasks with an obstacle clearance or a moving context (`t1_belt`, `t1_occ`, `t3_phase`, `t3_full`, and the four capture tasks and `t5_replan`).

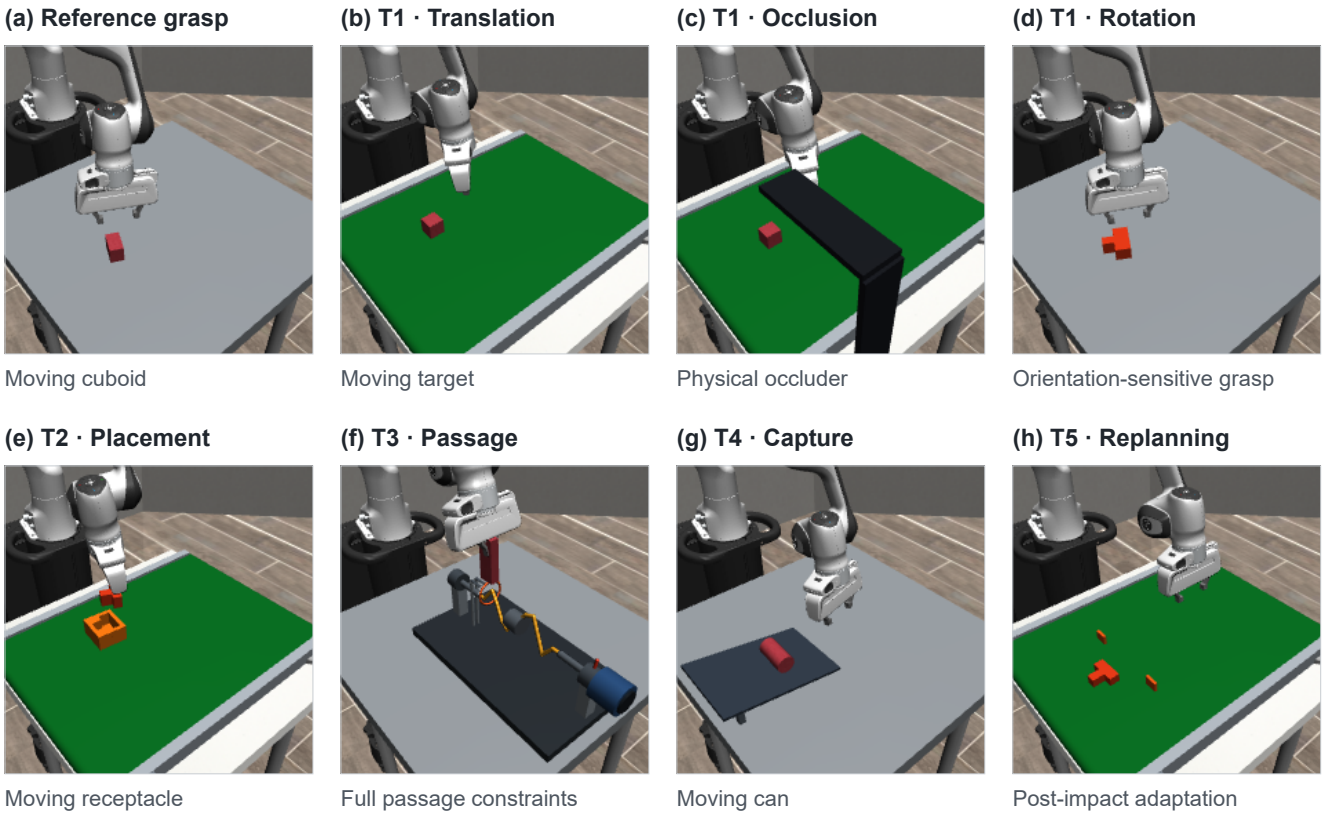


Figure S2. Dynamic-14 observations. Eight examples cover reference grasping, grasping a translating, occluded, or rotating target, placement into a moving receptacle, passage along a rotating wire, capture of a rolling object, and grasping after an impact. Each panel is the final observed, uncropped 224×224 agent-view RGB of one development scene. Table S3 lists all fourteen tasks.

Terminal events. We check success at every 2 ms physics step, after the ordered requirements of Table S3, and an interrupted continuous condition resets its counter. Terminal events fall into five categories. *Invalid* events are corrupted simulator states. *Boundary* events are a workspace or downstream exit, a drop, or an expired goal window. *Safety* events are unsafe contact, an excessive rim impulse, wire contact, a release impulse, early robot contact, unsupported tray contact, or a striker trap. *Task violations* are a release outside the goal, a path deviation or backtracking, a premature release, a deflection without capture, tipping, a missing or extra impact, or an insufficient change of outgoing motion. The last category is the *timeout*. Each family evaluates its events in a fixed order per physics step and keeps the first terminal event.

B.2. Scenarios and Demonstrations

We sample continuous ranges uniformly and discrete choices with equal probability, and then apply physical-feasibility checks. Evaluation replays the stored accepted scenarios. Harmonic translation follows $\mathbf{p}(t) = \mathbf{c} + (0, v_y, 0)t + (A_x \sin(2\pi f_x t + \phi), 0, 0)$ with $\phi \sim \mathcal{U}[-\pi, \pi]$. Capture targets start at rest and move under gravity and friction on ramps of $[5, 15]^\circ$. We accept an occluded scene only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames.

The training pool contains 11,200 demonstrations, 800 per task, generated by experts with privileged simulator state; the learned policies receive only the inputs of Sec. 3. We record the demonstrations at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry, from which we compute the future labels, their validity masks, and the command labels.

B.3. Execution Protocol

Executor. Every method runs under the same asynchronous executor on a 50 ms control clock, and queries start on its period boundaries. Let L_t be the total latency of query t, the time that physics advances while it runs: its inference time under the real-time protocol plus any added delay. The executor indexes each chunk by time from capture, so that its j-th command executes at

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S1}$$

Table S3. Success conditions of the fourteen Dynamic-14 tasks, with the thresholds of our task implementations. A task must pass its preceding stages and avoid terminal failures. All tasks run at 20 Hz with a 2 ms physics step; durations below refer to physical time. COM denotes center of mass.

Task	Ordered success conditions	Limit
grasp; t1_belt, t1_xyz, t1_so3, t1_occ	Approach within 0.10 m; bilateral fingerpad contact for 0.05 s; collision-geometry bottom at least 0.05 m above support; retain contact and clearance for 0.50 s. T1 adds conveyor drive, translation, rotation, or a physical occluder. SO3 has no additional success-angle threshold; unsafe occluder contact terminates OCC.	160 steps 8 s
t2_trans	Approach within 0.10 m. Entire object collision geometry inside the receptacle: planar clearance and floor gap both ≥ -0.002 m, top no higher than the opening. Release requires an opening command < -0.5 and no fingerpad contact. Relative linear/angular speeds ≤ 0.08 m/s and 0.8 rad/s for 0.10 s, followed by a 0.50 s stable dwell.	200 10 s
t2_se3	Approach within 0.05 m and 0.12 rad. Full T-shaped containment: planar clearance ≥ 0.002 m, floor gap in $[-0.002, 0.002]$ m, tilt ≤ 0.05 rad, top below opening. After release, relative speeds ≤ 0.02 m/s and 0.04 rad/s for 0.10 s, then a 0.50 s dwell. The 0.12 rad gate applies to approach.	200 10 s
t3_phase	The episode starts with the wand already grasped. Phase alignment: distance ≤ 0.090 m and loop-direction error ≤ 0.20 rad. Enter within 0.030 m at progress ≤ 0.15; reach historical maximum progress ≥ 0.60, then ≥ 0.85 and endpoint distance ≤ 0.055 m.	420 21 s
t3_full	Acquire the wand with bilateral contact for 0.05 s, then complete the phase stages above. In a later sample, retain current progress ≥ 0.85 and move at least 0.045 m from both path and endpoint. Finally open (< -0.5) with no fingerpad contact. Releasing before exit fails.	640 32 s
t4_sphere	Approach within 0.10 m; first qualifying contact after ramp exit; any tray contact before success fails. Bilateral hold for nominal 25 ms (12 physics samples, 24 ms). Raise COM by 0.05 m from its height when the bilateral-hold criterion is first met, and retain contact and height for 0.50 s.	120 6 s
t4_sphere_tabletop; t4_slider	Approach within 0.10 m; contact after ramp exit, while supported, in the capture region, and moving at planar speed ≥ 0.08 m/s. Bilateral hold for 0.05 s, COM lift ≥ 0.05 m from the first attainment of that hold criterion, then contact and height for 0.50 s.	120 6 s
t4_can	Same supported-capture sequence. Before first fingerpad contact, a can-axis absolute dot product with vertical > 0.45 fails as unstable tipping. The configuration does not require a transverse closing axis.	120 6 s
t5_replan	Exactly one selected-side impact group completes before robot contact. At 30 ms after the physical impact-end timestamp, branch-signed lateral speed ≥ 0.035 m/s and yaw rate ≥ 0.35 rad/s. Approach within 0.10 m; bilateral hold with grasp-axis error $\leq 18^\circ$ for 0.05 s; COM lift ≥ 0.05 m from the first attainment of that hold criterion and stable hold for 0.50 s. Impact-group lag is 12 ms and maximum contact is 0.10 s.	120 6 s

where d_{task} is a command delay built into a task (50 ms for the post-impact grasp, zero otherwise). The chunk takes over after $k_t\Delta_a$ with $k_t = \lceil L_t/\Delta_a \rceil$: its commands $j < k_t$ have expired and are discarded, it replaces the remainder of the previous chunk from command k_t on, and the next query starts at that same boundary. Query t therefore executes its commands $j = k_t, \ldots, \min(8, k_t + k_{t+1}) - 1$, one per control period, and a method whose inference fits in one control period replans before every command. The time-to-action that we report is the measured latency before this rounding (Appendix B.7). Since $j \leq 8 - 1$, every executed command lies at most $350\,\text{ms} + d_{\text{task}}$ after capture and thus within the 400 ms of the forecast supervision. When $k_t + k_{t+1} > 8$, e.g., for a constant total latency above 200 ms, a chunk runs out before the next one arrives, and if $k_t \geq 8$, all its commands have expired. Before the first chunk of an episode arrives, and whenever a chunk runs out before the next one, the controller holds the current arm target and gripper command while physics advances; no command is repeated.

Real-time protocol. Unless noted otherwise, physics advances during inference by the method's native latency, drawn per query from its latency distribution measured at batch size 1 on one RTX 5090 (Appendix B.7). The ACT/DP rollouts run on RTX 5090; the other rollouts use RTX PRO 6000. Their own compute time does not enter the episode. Table S14 reports three alternatives. With *paused* physics, the executor is unchanged and the simulation waits for inference; an added delay still advances physics. Without added delay, $L_t = 0$, and command 0 executes at capture. With *equal latency*, every method is charged the p95 latency

Table S4. **Roles and tolerances of the Dynamic-14 tasks.** The target is the body whose pose error enters a contact or containment tolerance ε. The context comprises obstacles, avoided with clearance m, and drivers that move the target. Speeds follow the sampling ranges of Table S5 (capture tasks: at the ramp exit). Terms: error terms that a task exercises (Eq. 2 and Appendix E). v: moving target. $r\theta$: orientation enters the contact. e_{dyn}: non-constant motion (harmonic, gravity, friction, impact). m: a context body must be avoided. Occ.: history gaps. Delay: command delay built into the task.

Task	Code	Target	Context	ε (mm)	$\|\mathbf{v}\|$ (m/s)	m (mm)	Terms
Harmonic grasp	grasp	block	table	[X]	0.03–0.20	–	v, $r\theta$, e_{dyn}
Conveyor grasp	t1_belt	part	belt	[X]	0.04–0.10	–	v
Translating grasp	t1_xyz	part	table	[X]	0.04–0.15	–	v, e_{dyn}
Rotating grasp	t1_so3	part ($\omega \leq 1.2$ rad/s)	table	[X]	0.04–0.15	–	v, $r\theta$, e_{dyn}
Occluded grasp	t1_occ	part	occluder canopy	[X]	0.04–0.10	[X]	v, m, occ.
Moving receptacle	t2_trans	receptacle	table	[X]	0.03–0.06	–	v
Rotating receptacle	t2_se3	T-shaped receptacle	table	[X]	0.03–0.06	–	v, $r\theta$
Wire passage	t3_phase	wire loop (entry)	rotating wire	30	[X]	swept	v, $r\theta$, m
Wire passage, full	t3_full	wire loop (entry)	rotating wire	30	[X]	swept	v, $r\theta$, m
Ball capture	t4_sphere	ball	tray, ramp	[X]	[0.4–0.7]	[X]	v, e_{dyn}, m
Ball capture, table	t4_sphere_tabletop	ball	table, ramp	[X]	[0.4–0.7]	–	v, e_{dyn}
Slider capture	t4_slider	slider	table, ramp	[X]	[0.4–0.8]	–	v, e_{dyn}
Can capture	t4_can	can	table, ramp	[X]	[0.4–0.7]	–	v, $r\theta$, e_{dyn}
Post-impact grasp	t5_replan	part	striker	[X]	0.12–0.14	[X]	v, $r\theta$, e_{dyn}, m, delay

of SE(3)-WAM regardless of its own speed. With *original executors*, each baseline keeps the execution scheme of its paper, e.g., a longer chunk before replanning.

Added latency. An added latency of 0, 50, 100, 150, or 200 ms is inserted after inference on top of the protocol's latency, and physics advances during it.

Pose noise. With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

B.4. Baselines

Common conditions. Table S6 summarizes the adaptations. Each baseline keeps its official architecture, action chunk, and initialization. ACT and Diffusion Policy train from scratch, $\pi_{0.5}$ is fine-tuned from `pi05_base`, and the other methods start from their released weights. All of them train on the same 11,200 demonstrations, whose commands we map to each action space through the same Cartesian controller. The seven trained baseline endpoints use agent-view RGB only, as does SE(3)-WAM. The native missing-view paths are retained: $\pi_{0.5}$ masks its two zero wrist slots; DreamVLA uses a preprocessed black wrist image with wrist auxiliary supervision masked; OpenWAM-α and DynamicWAM fill their missing wrist regions with black. Each baseline follows its official update budget, trained once with seed 0; the input and action adaptations are stated here and in Table S6. For DreamVLA, we ran both its original 40-epoch schedule and a 20-epoch cosine schedule and report the one that was better on development scenes.

Inputs. PUMA [13] receives its historical optical flow, computed on the rendered frames. DreamVLA receives seven causal RGB frames, the end-effector position and orientation, the finger positions, and a fixed task text, and it samples with ten DDIM steps and classifier-free guidance of 1.5. OpenWAM-α receives the current RGB views, joint values, gripper opening, and task text, and it generates video and actions jointly with video decoding disabled. DynamicWAM receives its flow history and kinematic descriptors.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags and masking. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them; for DynamicWAM, they join its kinematic descriptors. The output layer of the MLP is zero-initialized, and training follows the base model's recipe.

B.5. Splits and Statistics

We select configurations on 1,400 development scenes (100 per task). All Dynamic-14 results use a separate held-out set of 1,400 scenes generated with the same ranges and new

Table S5. Scenario proposal ranges. Length, linear velocity, angle, and angular velocity use m, m/s, rad, and rad/s unless marked otherwise. Positions are in world coordinates except the stated ramp-local offsets. Continuous intervals are uniform proposals; discrete choices are equiprobable before feasibility checks. Evaluation replays the stored accepted scenarios, whose distribution need not remain uniform. Harmonic centers are not initial positions.

Task	Initial-state proposal	Motion or event parameters
grasp	Harmonic center $c_x \in [-.08, .08]$, $c_y \in [-.26, -.19]$, $z = .822$; yaw $[-1.1, 1.1]$.	$v_y \in [.025, .125]$, $A_x \in [.015, .035]$, $f_x \in [.3, .7]$ Hz; $\omega_z \in [-1.2, 1.2]$.
t1_belt	Target $x \in [-.08, .08]$, $y \in [-.26, -.18]$, $z = .822$; identity orientation, zero initial twist.	Belt $v_y \in [.04, .10]$.
t1_occ	Same target proposal as Belt; canopy at $(.322, -.29, 1.20)$.	Same belt range; clearance parameter .20; physical occlusion.
t1_xyz	Harmonic center $c_x \in [-.025, .025]$, $c_y \in [-.24, -.20]$, $z = .818$; yaw $[-.35, .35]$.	$v_y \in [.04, .09]$, $A_x \in [0, .03]$, $f_x \in [.25, .65]$ Hz; $\omega_z = 0$.
t1_so3	Same as XYZ.	Same translation; $\omega_z \in [-1.2, -.4] \cup [.4, 1.2]$, equal sign probabilities.
t2_trans	Goal $x \in [-.13, -.08]$, $y \in [-.18, -.12]$, $z = .825$; fixed orientation.	Goal $v_y \in [.03, .06]$; no rotation.
t2_se3	Same goal x, y range, $z = .804$; yaw $[-.3, .3]$.	Goal $v_y \in [.03, .06]$; $\omega_z \in [-.25, -.08] \cup [.08, .25]$, equal sign probabilities.
t3_phase	Wand $x, y \in [-.03, .03]$, $z = .870$, yaw $[0, 2\pi]$; wire phase $[-.25, .35]$.	Wire center $(.10, -.02, .94)$; world-X rotation $\omega_x = -\{2.5, 3, \ldots, 7.5\}^\circ$/s (11 equal-probability values).
t3_full	Wand at $(-.05, -.02, 1.04)$ with identity orientation; same wire phase range.	Same wire rotation; cradle and release requirement enabled.
t4_sphere	Ramp-local offsets: lateral $[-.015, .015]$, longitudinal/normal $(-.145, .030)$; lip $(.06, -.02, 1.10)$.	Gravity-driven from rest; gate release at tick 0; slope $[5, 15]^\circ$; target/ramp sliding friction $[.7, .9]$; unsupported capture.
t4_sphere_tabletop	Lateral $[-.015, .015]$, longitudinal/normal $(-.124, .034)$; lip $(0, -.06, .808)$.	Same slope and release; target/ramp friction $[.7, .9]$, table friction .35.
t4_slider	Lateral $[-.015, .015]$, longitudinal/normal $(-.129, .024)$; supported ramp.	Same slope and release; target/ramp friction $[.025, .045]$, table friction $[.012, .018]$.
t4_can	Lateral $[-.010, .010]$, ramp-axis yaw $[-4, 4]^\circ$; longitudinal position follows gate geometry and oriented can extent.	Same slope and release; target/ramp friction $[.25, .45]$, table friction $[.09, .15]$.
t5_replan	$x \in [-.005, .005]$, $y = -.29$, $z = .812$; yaw $[-.08, .08]$; zero initial twist.	Belt $v_y \in [.12, .14]$; equal-probability left/right striker triggered at $y = -.13$; stroke .098, pulse .35 s, actuator scale $[1, 1.1]$, force cap 4 N; command delay 50 ms.

source seeds that appear in no training, validation, or development episode; the first 100 feasible scenes in increasing seed order form each task's set. The full model and the five ablation controls use training seeds 0, 1, and 2, and each baseline uses seed 0. On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR, mostly on the wire passage, whose success ranged from [0] to [71]% across seeds. We report each training seed and the cross-seed mean and standard deviation; for the ablations, we also repeat the comparison without the phase-only wire passage (`t3_phase`). For paired 95% intervals, we average the seeds per scene and bootstrap held-out scenes within each task, jointly across methods. These intervals quantify scene uncertainty for the trained weights, and we treat differences whose interval lies within ± 2 points as ties.

B.6. Metrics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{100}{14} \sum_{k=1}^{14} \frac{1}{S_m n_k} \sum_{s=1}^{S_m} \sum_{i=1}^{n_k} y_{mski}, \qquad \text{(S2)}$$

for binary outcomes y_{mski} of method m, trained with seed s, on scene i of task k, with $S_m = 3$ training seeds for our variants and $S_m = 1$ for the baselines. The *position error at first contact* e_p is the distance of the end effector from the contact frame at the first contact. The *contact-time error* e_t is the distance from the first-contact time to the feasible contact window of the scene, and it is zero inside the window. The window is the set of control-period boundaries t at which the privileged expert could make the contact of the task's

Table S6. **Baseline adaptations.** All methods train on the same 11,200 demonstrations with the inputs listed below and run under the shared executor of Appendix B.3. Updates × batch follows the official recipe. Exec.: the execution scheme of the original paper (commands executed per chunk before replanning), used only for the original-executor control of Table S14. Params: trainable parameters, frozen encoders excluded. Seeds: training seeds.

Method	Initialization	Inputs	Updates × batch	Exec.	Params	Seeds
ACT [56]	scratch	current agent RGB, robot state	50,000 × 32	temporal agg.	[X]	1
Diffusion Policy [11]	scratch	two causal agent RGB frames, robot state	50,000 × 32	8	[X]	1
$\pi_{0.5}$ [37]	`pi05_base`	current agent RGB, robot state, task text; wrist slots masked	30,000 × 256	10	[X]	1
$\pi_{0.5}$ + SE(3)	`pi05_base`	planned: $\pi_{0.5}$ + SE(3) tokens; camera inputs pending	30,000 × 256	10	[X]	1
DynamicVLA [49]	released DOM ckpt.	frames $[-2, 0]$, robot state, task text	50,000 × 44	streaming	[X]	1
PUMA [13]	official checkpoint	RGB, optical-flow history, robot state, task text	[X]×[X]	[X]	[X]	1
DreamVLA [54]	released components	seven causal agent RGB frames, robot state, task text; black wrist input	48,220 × 512	1	[X]	1
OpenWAM-α [46]	official foundation	current agent RGB, joints, gripper, task text; missing views black	34,490 × 192	32	[X]	1
DynamicWAM [32]	official initialization	RGB, flow history, kinematic descriptors	40,000 × 128	16	[X]	1
DynamicWAM + SE(3)	official initialization	planned: DynamicWAM + SE(3) tokens; camera inputs pending	40,000 × 128	16	[X]	1
SE(3)-WAM	frozen MAE encoder	current agent RGB-D, SE(3) and robot histories	51,740 × 512	1	523.8M	3

first success stage. A time t belongs to it when the expert's contact frame on the target at t, taken from a rollout of the same scene with the robot held at its initial pose, (i) satisfies the geometric conditions of that stage (Table S3), (ii) has an inverse-kinematics solution within the expert's joint limits, and (iii) can be reached from the initial end-effector pose by t under the expert's per-command limits of 0.05 m per translation axis and 0.5 rad per rotation-vector component. The earliest such time is the larger of $\lceil \max_i |\Delta p_i| / 0.05\,\mathrm{m} \rceil$ and $\lceil \max_i |\Delta \phi_i| / 0.5\,\mathrm{rad} \rceil$ control periods, a lower bound that ignores the controller's tracking lag. In this reference rollout, only robot–target collisions are disabled. The window depends only on the scene and is shared by all methods. The two placement tasks start with the object held, so their contact is the first contact of the held object with the receptacle, and their contact frame is the end-effector pose implied by the expert's placement target. The phase-only passage (`t3_phase`) starts with the wand held and succeeds without any contact, so the averages of e_t and e_p cover the other 13 tasks, while SR covers all 14. We report both errors on episodes that make contact; failures without contact stay in the SR denominator. We distinguish three times. The *time-to-action* is the measured wall-clock time from the arrival of an observation to the availability of its first command, without rounding; the *handover time* $\ell_t = k_t \Delta_a$ rounds the total latency up to the control period (Appendix B.3); and the *action age* is the execution time of a command minus the capture time. Table S11 reports e_p together with the number of episodes that make contact.

B.7. Timing

We time all methods on one RTX 5090 (32 GB), the GPU of our robot, with exclusive use and batch size 1. These measurements form the latency distributions that the simulated rollouts replay independently of the rollout hardware. For each method, each of three rounds uses thirty warm-up calls and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and a rotated method order. Each method uses the precision of its release; SE(3)-WAM uses FP32. The time-to-action includes encoding the image and generating the first command, and on the robot also tracking the newest frame. For SE(3)-WAM, we time the forecast heads separately, although deployment skips them. Table 1 gives the p95 values and Table S6 the parameter counts. On the robot, traces record capture, availability, preprocessing, the start and end of inference, queue entry, command issue, and execution. We report the p50, p95, and maximum action age. A query misses its deadline when its time-to-action exceeds the 50 ms control period, so that the next period executes a remaining command or holds; we report the fraction of all queries of the test trials that miss it, together with the peak memory (Table S7).

Table S7. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Image encoding	—	—
World stream (once per query)	—	—
Action stream (5 Euler steps)	—	—
Time-to-action (total)	—	—
Forecast heads (training only, for reference)	—	—
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C. Real-Robot Setup

C.1. Platform and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]. The camera records RGB-D at [resolution] and [frame rate], and we calibrate its extrinsics by [procedure] before each session. SE(3)-WAM uses this fixed third-person view; baselines whose recipes require a wrist view receive [per-method camera configuration]. All policies run on one RTX 5090. Table S8 defines the six tasks. T1 and T2 share a belt conveyor, T3 uses a turntable, T4 a remotely driven toy car, T5 a cue-driven collision between two balls, and T6 a ramp. Touching or pushing the target without completing the task, a single-finger hold, and an action after the target has left the workspace count as failures; [equipment-invalid trials and safety interventions are repeated or counted as failures by the following rule].

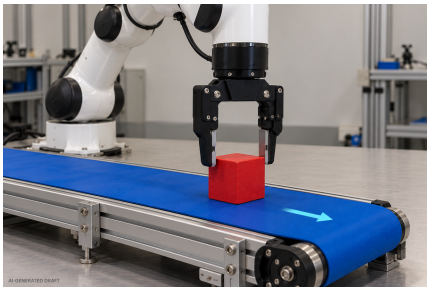
C.2. Pose Tracking

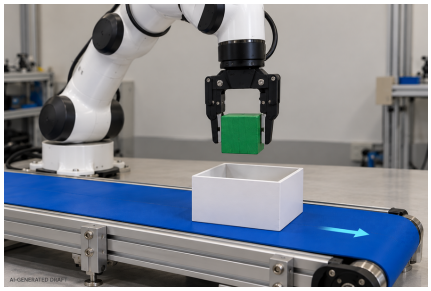
Tracking is marker-free. Grounding DINO [29] localizes the object named by the task in the first frame, SAM 2.1 [40] gives its initial mask, and FoundationPose [47] initializes the object's 6D pose from this mask and then tracks it. [FoundationPose variant and mode (model-based with an object mesh or model-free with reference views), how the object frame is kept consistent across demonstrations and trials, and re-initialization after a loss.] The tracker runs on the control clock: at every control-period boundary, it updates the pose on the newest camera frame captured at or before the boundary, so the pose history has the 50 ms spacing used in training, and each estimate keeps the capture time of its frame. A query waits for the update of its own boundary and then runs the policy on the same GPU. While a query spans further boundaries, the tracker keeps updating at a higher GPU priority and processes only the newest frame, so that an update never queues behind older ones; the tracking time and this contention are part of the measured latency (Table S7). [Camera frame rate and the measured time per update.] When the target is lost, the estimate is marked invalid, and the previous pose is not carried forward. [How the target poses of the demonstrations, used as inputs and training labels, are obtained.] For the ball tasks, the target-state tokens keep the translation, velocity, and acceleration of the ball center and mask its orientation and angular motion, and the pose head is supervised on translation only. Table S9 reports the tracking accuracy.

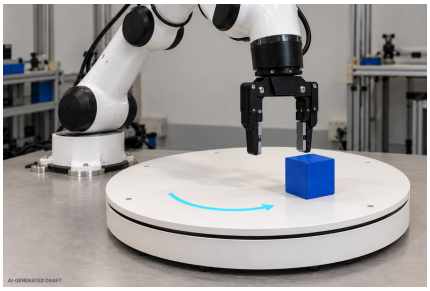
C.3. Trials

We compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and Diffusion Policy [11], with 20 trials per task and method. All methods use the same pre-generated trial keys, which fix the object instance, the initial pose, and the motion parameters, sampled by [sampling protocol]. Trials run in [randomized method order], so that drift of the apparatus does not align with the methods. An operator resets the scene. A trial starts at the first usable observation, the robot may move at any time, and a trial ends at success, failure, or a timeout of [X] s. Demonstrations, development trials, and test trials come from separate sessions. SE(3)-WAM is fine-tuned per task from the DROID base (Appendix A.5), and the baselines are trained or fine-tuned on the same demonstrations with [per-method initialization and training settings]. For every method, we log the end-to-end time-to-action of each query on the same RTX 5090, including tracking for SE(3)-WAM; Table 4 reports its p95. Table S10 reports successes per task, 95% binomial intervals for the pooled rates, and paired differences from a bootstrap over trial keys.

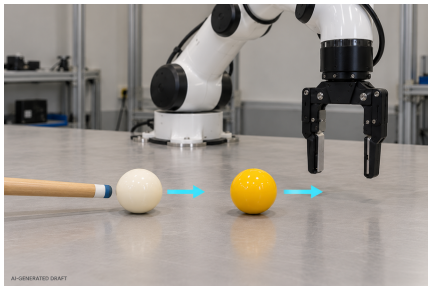
Fig. S4 shows typical failures of SE(3)-WAM.


T1: Conveyor grasping


T2: Moving-receptacle placement


T3: Turntable grasping


T4: Moving-car grasping


T5: Post-collision interception

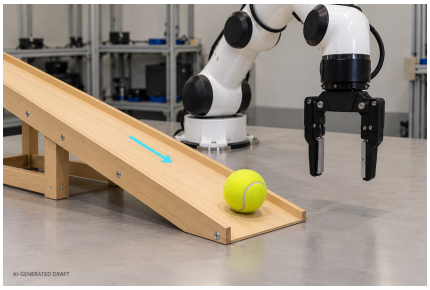

T6: Ramp interception

Figure S3. **Real-robot tasks.** AI-generated concept illustrations, to be replaced by photographs of the setup; arrows indicate the intended target motion. T6 shows the rolling variant; the task also includes sliding objects.

Table S8. Real-robot task definitions. $N = 100$: training demonstrations per task; 20 additional demonstrations per task are reserved for validation.

Task	Device, motion, and objects	Operating range; N	Success and failure criteria
T1	Conveyor transports a graspable cube.	[belt speed]; 100	[stable grasp and lift; failures]
T2	Conveyor transports an open box; the arm places a cube into it.	[belt speed]; 100	[placement and retention; failures]
T3	Turntable carries a cube around its axis.	[angular speed, radius]; 100	[stable grasp and lift; failures]
T4	Remotely driven toy car moves on the tabletop.	[speed, path]; 100	[car grasp and lift; failures]
T5	A cue-driven ball strikes a second ball; the arm intercepts the second ball after the collision.	[impact and outgoing speed]; 100	[post-impact capture; failures]
T6	A ball rolls or a block slides down a ramp toward the interception region.	[slope, release, speed]; 100	[capture and retention; failures]

Table S9. **Real-robot pose tracking.** Median/p95 translation (mm) and rotation (°) error of the tracker against a marker-free reference, the uncertainty of that reference, and the fraction of lost observations. Ball rotation is not used.

Target	mm	°	Ref. uncertainty	Lost
Real rigid object	—	—	—	—
Real ball	—	–	—	—

Figure placeholder: Fig. S — failures of SE(3)-WAM on the real robot

Rows: two or three typical failures of SE(3)-WAM ([tasks]), chosen as the most frequent failure categories of Table S10. **Columns:** the query frame at τ_t, the frame at the execution time τ^a of the first executed command, and the frame at which the trial fails. **Overlays:** blue tracked past poses, purple commanded gripper path; a timeline strip under each row; the failure criterion and a one-sentence cause at the right end of each row (e.g., tracking loss under occlusion, a bounce that the history did not predict). **Rules:** real photos, unretouched; overlays from logged data.

Figure S4. **Failures of SE(3)-WAM on the real robot.** [One sentence per row on the cause of the failure.]

Table S10. **Real-robot successes** per task (successes/trials, 20 trials per task; tasks in Table S8). All: pooled success rate (%) with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys.

Method	T1	T2	T3	T4	T5	T6	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	–

D. Additional Results

D.1. Per-Task Results and Outcomes

Table S11 breaks the held-out Dynamic-14 results of Table 1 down by task family and reports the position error at first contact with its episode count, and Table S12 breaks them down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task and the contact-time error of each method. Table S13 reports how episodes end, so that a method that trades one failure for another, e.g., fewer collisions for more timeouts, becomes visible. Every failure stays in the SR denominator.

With native latency, ACT succeeds in 588 of 1,400 scenes: 42.0% (95% scene bootstrap interval 39.9–44.1%). Its task-balanced contact errors are $e_t = 0.412$ ms [0.278, 0.551] and $e_p = 32.2$ mm [31.5, 32.9], from 1,109 episodes with contact across the 13 eligible tasks. These intervals use 10,000 bootstrap resamples within each task and one trained checkpoint. None of its 191,475 queries exceeds 50 ms; the p50, p95, and maximum action ages are 50, 50, and 100 ms. The 100 ms ages occur in the post-impact task, whose actuator adds one control period.

[Tasks on which every method fails, if any, and why.] [Tasks on which a baseline outperforms SE(3)-WAM.] Most remaining failures of SE(3)-WAM are [the dominant failure mode].

D.2. Protocols and Latency

Table S14 evaluates every method under three further protocols. Paused physics shows how the relative performance changes when the cost of computation is ignored, equal latency compares all methods at the p95 latency of SE(3)-WAM, and the original executors check whether the shared executor removes a capability that a baseline relies on. With paused physics, the difference between SE(3)-WAM and [name] is [±X.X] points, and under equal latency it is [±X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S15 gives the success at every added latency; its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

D.3. DOMINO

On DOMINO [13], we compare SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of its clean level-1 setting. All methods use the benchmark's demonstrations, scenes, and native evaluator, one training seed, and 50 episodes per task and method. We report SR and the manipulation score (MS), which scales route completion by penalties for leaving the workspace or view and for collisions (Table 2). SE(3)-WAM uses domain-specific normalization and reads the simulator pose of the target named by the instruction; slots without a valid pose are masked as in Appendix A.1. The RGB-based baselines do not receive this pose. [Summarize the DOMINO results.]

Table S11. **Success by task family and contact error** on held-out Dynamic-14 scenes with native latency. SR (%) per family, task-balanced within the family. e_p: position error at first contact (mm), averaged over the 13 tasks with a contact; n: number of episodes with a first contact, over which e_p is computed.

Method	Grasp	Place	Pass	Capture	Impact	e_p	n
ACT	33.0	13.0	49.5	63.5	44.0	32.2	1109
Diffusion Policy	—	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—	—
PUMA	—	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—

Figure placeholder: Fig. S3 — pose noise

x: clean poses, nominal correlated noise, twice its amplitude, and nominal noise with missing observations; y: SR of SE(3)-WAM per task family, frozen weights, real-time protocol; paired intervals over matched scenes.

Figure S5. **Robustness to pose noise.** SR of SE(3)-WAM under synthetic perturbations of the target pose, relative to clean poses.

D.4. Pose Noise

Fig. S5 evaluates the final model with frozen weights under the three synthetic pose perturbations of Appendix B.3, on the same held-out scenes at zero added latency and with all three training seeds. Relative to clean poses, SR changes by [X.X], [X.X], and [X.X] points under nominal correlated noise, twice its amplitude, and missing observations.

D.5. Design Screening

We screened the encoder, the history representation, the forecast targets, and the coupling of world and action streams with smaller models: 4 layers of width 768, trained for 4 epochs (40,104 updates at a global batch size of 16) and evaluated with the synchronous executor that preceded our real-time protocol. Each group keeps its own encoder and seed count, stated below, and conditions are compared within a group. These studies selected the components of the final model; the four ablations of Sec. 4.3 then test its central components at full scale.

Table S12. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 1, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. σ: SE(3)-WAM with nominal synthetic correlated pose noise (Appendix D.4). Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. The last row gives the contact-time error e_t (Appendix B.6). Task codes are listed in Table S4.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	σ	**Ours**	Δ [95% CI]
Harmonic grasp	48.0	—	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	46.0	—	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	18.0	—	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	21.0	—	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	32.0	—	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	26.0	—	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	0.0	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage	99.0	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	0.0	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture	36.0	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	71.0	—	—	—	—	—	—	—	—	—	—	—	—
Slider capture	57.0	—	—	—	—	—	—	—	—	—	—	—	—
Can capture	90.0	—	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	44.0	—	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	42.0	—	—	—	—	—	—	—	—	—	—	—	—
e_t (ms)	0.412	—	—	—	—	—	—	—	—	—	—	—	—

Table S13. **Outcome categories** on held-out Dynamic-14 scenes (% of episodes, seed-averaged, real-time protocol). Categories follow Appendix B.1, and rows sum to 100 before rounding. The noisy-pose row reuses the nominal synthetic correlated-noise episodes from the robustness study.

Method	Success	Safety	Task	Bound.	Timeout	Inval.
ACT	42.0	20.5	3.9	16.1	17.6	0.0
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, noisy poses	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

Table S14. **Protocol controls** on held-out Dynamic-14 scenes (SR, %). Paused: shared executor, with physics stopped during inference. Equal latency: shared executor, with every method charged the p95 latency of SE(3)-WAM. Original executor: real time with each method's own execution scheme. Real time: the protocol of Table 1. Late: queries in real time whose time-to-action exceeds the 50 ms control period (%).

Method	Paused	Equal latency	Original executor	Real time	Late (%)
ACT	—	—	—	42.0	0.0
Diffusion Policy	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM	—	—	–	—	—

Encoder and backbone. We compared a CNN trained with the policy, frozen DINOv3 ViT-S/16 and ViT-L/16 [41], and MAE ViT-B/16 [18] (three seeds for the MAE reference; Table S16), and selected MAE. We also compared a backbone with shared attention and stream-specific feed-forward networks against one with fully separate world and action parameters, at about 47.3M backbone parameters and with three seeds each ([source of the backbone comparison]), and selected the shared attention.

History. With a frozen DINO-L encoder and one seed per condition, we compared the current state alone, the target-motion history, RGB windows, Farnebäck flow [14], flow with motion statistics [32], codec motion vectors, video-VAE latents, and compressed windows (Table S17). Table S18 replaces the past target poses of the trained history model by its current pose at test time.

Table S15. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	42.0	—	—	—	—
Diffusion Policy	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—

Table S16. **Observation encoders** at the screening scale (Appendix D.5). Params: encoder parameters; p95: time-to-action.

Encoder	Params	Trained	SR	SR_{150}	p95
CNN	—	yes	—	—	—
DINOv3 ViT-S/16	22M	frozen	—	—	—
DINOv3 ViT-L/16	300M	frozen	—	—	—
MAE ViT-B/16*	86M	frozen	—	—	—

Table S17. **History representations** at the screening scale (frozen DINO-L encoder, one training seed, 1,400 scenes). The first eight conditions receive the current target pose; the last adds the target-motion history.

History input	Success / 1,400	SR (%)
Current image	748	53.43
RGB window	662	47.29
Optical flow	720	51.43
Flow + motion statistics	723	51.64
Codec motion vectors	733	52.36
Video-VAE latent	710	50.71
Compressed window (age)	634	45.29
Compressed window (uniform)	670	47.86
Current image + target-motion history	824	58.86

Table S18. **Stationary-history intervention** on the history model of Table S17 (1,400 scenes). At test time, past target poses are replaced by the current pose and the kinematic features are recomputed.

Target history	Success / 1,400	SR (%)
Unchanged	824	58.86
Stationary replacement	9	0.64

Forecast targets. With a frozen DINO-L encoder and one seed per condition, we compared fifteen future targets (Table S19), from target-only and image-space forecasts to role-aware combinations (Table S20). A second group with the MAE encoder and one seed removed each future appearance target from the selected configuration. In this group, no future target improved clearly on the variant without future supervision (P00, 62.78% against 62.38% for scene flow and SE(3)); single seeds at this scale do not resolve such differences, and the full-scale ablations of Sec. 4.3 test future supervision in the final model.

Coupling. With the MAE encoder and three seeds per condition, we compared six couplings of the world and action streams. R0 disables every forecast loss. R1 adds one object query and one query per patch to the world stream, which attend to $\mathcal{S}$, $\mathcal{M}$, and each other and carry the forecast readouts, without any edge to the actions. R2 lets the action tokens attend to these queries at every layer, R3 lets the queries attend to the actions, and R4 opens both edges; in R3 and R4, the world states are recomputed at every Euler step. R5 decodes the forecast once per query and encodes the target trajectory and the flow with visibility, with rotations in a continuous 6D form, through a geometry encoder (0.37M parameters) into tokens that the actions attend to. Future supervision without any edge to the actions (R1) exceeded R0 by 3.14 points on average, with seed-wise differences of -1.29, +9.64, and +1.07. Feeding the forecasts to the actions gave no consistent further gain: R2 and R5 differed from R1 by -1.07 and +0.36 points on average, with standard deviations of 4.28 and 3.45 points over the seed-wise differences, and the effects of both cross-stream edges changed sign across seeds (Table S21). We therefore use the forecasts only as training supervision, which also keeps deployment free of their cost.

D.6. Forecasts at Execution Time

Table S22 compares the learned SE(3) forecast of the final model offline with constant-velocity and constant-acceleration extrapolation from the same pose history.

Fig. S6 aligns the query, the forecast, the ground truth, the issued actions, and the first contact in physical time. The examples show where forecast errors arise, e.g., at an impact or when a rolling object leaves its ramp.

D.7. Supplementary Video

The supplementary video shows real-robot trials of SE(3)-WAM and the strongest baseline on all six tasks with the tracked poses overlaid, Dynamic-14 episodes of every task family with the forecasts of the training-time heads, and failure cases.

Table S19. Future-target definitions of the forecast screening. Shape excludes batch and query axes. Displacements are cumulative from the current state, except for codec vectors, which keep the correspondence of the decoder. All targets come from the recorded demonstrations. Future depth, DINO, and dynamic RGB are also the training-only auxiliary targets of the selected model.

Target	Shape	Support, units, and validity
Target pose	8×6	Cumulative translation (m) and left-relative rotation vector (rad). Requires a valid pose and query.
Scene flow, 14×14 grid	$196 \times 8 \times 3$	Fixed source grid, cumulative displacement in base-frame meters. Requires valid source depth and a finite future displacement.
Target tracks (2D)	$32 \times 8 \times 3$	Target-mask source points with two image-displacement channels and visibility; padded points are invalid. This is the P09 target, not a 3D rigid-pose fit.
Object DINO	8×1024	Mask-weighted pooling of future patch features with channel layer normalization. Requires a positive mask area.
Scene tracks (2D)	$196 \times 8 \times 3$	Grid source points with two image displacements divided by 224 and a visibility logit.
Dense 2D displacement	$8 \times 2 \times 224 \times 224$	Fixed source pixels, displacement divided by 224. Requires a valid source and a finite projection.
Codec vectors	$8 \times 2 \times 64 \times 64$	Each future/current pair is encoded as P/I frames, and vectors are divided by 64. Only valid vector regions count.
Video-VAE latent	$48 \times 2 \times 14 \times 14$	Frozen Wan2.2 VAE posterior mean with the VAE's own normalization. The two groups end at 200 and 400 ms.
Future depth	$8 \times 1 \times 224 \times 224$	Metric depth on finite positive pixels.
Future DINO	$8 \times 196 \times 1024$	Frozen DINOv3 ViT-L/16 teacher [41], 14×14 patches, channel layer normalization.
Dynamic RGB	$8 \times 3 \times 224 \times 224$	RGB divided by 255 on surfaces moving more than 1 mm, including the robot.

Table S20. **Forecast targets** at the screening scale. The first group uses a frozen DINO-L encoder and seed 0, with its 140 development scenes and the 1,260 remaining scenes reported separately. The second group uses the MAE encoder and seed 1 on 1,400 scenes; it keeps scene flow, SE(3), and the current-target RGB reconstruction and removes future appearance targets. Compare conditions within each group.

ID	Future supervision	Dev. success / 140	Success / 1,260	SR_{1260} (%)
P00	None	88	791	62.78
P01	3D scene flow	95	782	62.06
P02	SE(3)	92	789	62.62
P03	Scene flow + SE(3)	93	786	62.38
P04	Video-VAE latent + SE(3)	92	783	62.14
P05	Dense 2D displacement + SE(3)	87	746	59.21
P06	Codec motion + SE(3)	86	795	63.10
P07	Sparse scene 2D tracks + SE(3)	90	807	64.05
P08	Scene flow + object DINO	89	794	63.02
P09	Scene flow + target 2D tracks	88	808	64.13
A_α	P03 + motion magnitude	89	778	61.75
P12	P03 + depth	90	791	62.78
P13	P03 + DINO	92	788	62.54
P14	P03 + dynamic RGB	89	790	62.70
P15	P03 + depth / DINO / dynamic RGB	94	802	63.65
AUX0 MAE-B: future appearance auxiliaries		Success / 1,400		SR_{1400} (%)
None		894		63.86
Depth only		867		61.93
DINO only		946		67.57
Dynamic RGB only		926		66.14
All three (existing reference)		995		71.07

Figure placeholder: Fig. S4 — forecasts aligned with what happened

Rows 1–4: the median-error held-out episode of each motion type (conveyor, rotating target, rolling capture, post-impact); **row 5:** a failure. **Columns:** the query frame at τ_t; the frame at the execution time τ^{a} of the first executed command; and a 3D inset in the base frame. **Overlays:** blue past SE(3) poses; orange predicted target triads at $\tau_t + \delta_h$ and sparse flow arrows, read from the training-time heads; gray ground-truth poses at the same physical times; purple issued gripper path; a marker at first contact. A strip under each row plots translation and rotation error vs. horizon for learned SE(3), constant velocity, and constant acceleration. Annotate row 5 in red.

Figure S6. Forecasts of the training-time heads on held-out episodes, aligned in physical time with the ground truth and the issued actions. Rows 1–4 show the median-error episode of each motion type, and row 5 shows a failure.

Table S21. **Couplings R0–R5** at the screening scale (MAE encoder, three training seeds each, 1,400 development scenes per seed; Appendix D.5). SR: mean over seeds (%); s.d.: standard deviation over seeds (points).

Coupling	SR	s.d.
R0 (future losses off)	60.45	3.57
R1 (queries, no edge)	63.60	2.34
R2 (world→action)	62.52	1.97
R3 (action→world)	64.21	2.13
R4 (bidirectional)	63.00	4.19
R5 (decoded geometry)	63.95	1.91

Table S22. **Offline prediction errors.** Target-position error at 400 ms (mm) on the same held-out observations. CV and CA are analytic extrapolations from causal pose history; the learned head reads frozen full-model states.

	Const. vel.	Harmonic	Roll/slide	Impact
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
Learned pose head	—	—	—	—

E. Error Decomposition

This appendix derives the decomposition of Eq. 2 that motivates the design of SE(3)-WAM (Sec. 3.2). It describes how the errors of an estimate scale with the lead and the motion; it is not a model of success.

E.1. Translation

Let the true target position at the execution time be $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, where $\mathbf{v}$ is the velocity at τ_t and $\boldsymbol{\rho}$ the remaining motion, and let the policy's estimate be $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives the identity

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S3}$$

and the triangle inequality yields Eq. 2 with $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$ and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. The residual $\boldsymbol{\rho}$ is exactly the part of the motion that constant-velocity extrapolation misses: under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, so an acceleration estimate from the history removes part of e_{dyn}, whereas impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce. For a learned predictor, the split of its estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$ is not unique, so the decomposition states which information each term requires rather than how a particular network uses it.

E.2. Rotation and the Contact Point

With $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$ and $\hat{R}(\tau^{\mathrm{a}}) = \mathrm{Exp}(\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}})\hat{R}(\tau_t)$, rotations about a common fixed axis compose additively, and

$$\theta = \big\| \operatorname{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^{\top})^{\vee} \big\| \le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_\omega + \|\boldsymbol{\omega}\|\, e_\tau + e_\eta, \tag{S4}$$

with the current orientation error θ_{loc} and e_ω and e_η defined as for translation; for general rotations, the Baker–Campbell–Hausdorff formula adds terms of second order in the angles. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, has the world position $\mathbf{p} + R\mathbf{c}$, so its error is $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^{\top} - I)R\mathbf{c}$. Since the operator norm of $\hat{R}R^{\top} - I$ for a rotation by θ is $2\sin(\theta/2)$,

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}} - \mathbf{p}\| + 2r\sin(\theta/2) \le \|\hat{\mathbf{p}} - \mathbf{p}\| + r\theta. \tag{S5}$$

Orientation errors therefore enter the contact scaled by the lever arm, and a grasp or placement that depends on orientation needs the full SE(3) state of the target.

E.3. Context: Obstacles and Drivers

Obstacles. Suppose that the executed path keeps every point of the robot and the carried object at least m away from the *predicted* positions of the context points it passes. A collision at a context point then requires the true position of that point to differ from its prediction by at least m. Under this assumption, context errors below the clearance do not cause collisions, which is why a coarse estimate of the surroundings can suffice when $m \gg \varepsilon$. The assumption concerns the executed path: SE(3)-WAM does not enforce a margin explicitly, so the effect of coarse context supervision is tested empirically by the ablations (Sec. 4.3). A flow sampled on a grid can also miss structures thinner than its spacing.

Drivers. A driver, such as a conveyor, a ramp, or a striker, matters through the motion it imparts to the target. Its effect on the contact therefore enters Eq. 2 through the target's own terms, mostly through e_{dyn} when the driver changes the target's motion, as at an impact. A policy needs to know the driver's motion only well enough to anticipate when and how it will act on the target; the resulting motion of the target is what the dedicated SE(3) forecast supervises.

E.4. Scope

The decomposition addresses the approach to a rigid target whose motion is exogenous until contact. It does not cover deformable or articulated targets, whose motion is not determined by six numbers, or the motion after contact, which depends on the action itself and would call for action-conditioned prediction. Scenes in which the role of a body changes over time, e.g., sequential multi-object tasks, are also outside its scope.

F. Additional Related Work

This appendix extends Sec. 2 with work that the main text covers only briefly.

Dynamic manipulation and latency. Classical systems compensate latency with predictive tracking [4] and visual servoing [22], and catch or intercept moving objects by planning on estimated trajectories [2, 5, 19], later combined with grasp and reachability planning [1, 33]. Learned visuomotor policies [11, 56] and vision–language–action models [7, 8, 26, 36, 37, 61] generalize across objects and instructions, and most of them are trained and evaluated on scenes that stay still while the model computes. Among learned methods for moving scenes, DynamicManip synthesizes dynamic demonstrations from a single static one [27], and DynaWM refines the action chunk of a base VLA for moving objects [9].

World models and world–action models. World models have been used for planning with visual foresight [15], for reinforcement learning in a learned latent space [16, 17], and for planning on pretrained visual features [3, 58]. In manipulation, generated frames or subgoal images have been

converted into actions [6, 12, 55]. Studies of WAMs compare how the video and action streams should exchange information [46, 51] and how long and how compressed their context should be [21, 31]. Further forecast targets include motion-weighted video [23] and optical flow or motion vectors [10, 28].

3D and motion representations. Object-centric point clouds improve generalization [60], and 3D and 4D VLAs bring 3D and spatiotemporal inputs to large models [53, 57]. Dense pixel motion has also been predicted as an intermediate representation [35, 38, 39]. Beyond SE(3) decompositions, 3D dynamic scene memories [50] and scene flow estimation [30, 42] model how scenes move. Point trackers, pose estimators, and feed-forward reconstruction models [24, 44, 47, 48, 52] extract such motion labels from video; our DROID pretraining uses the 3D point tracks that PointWorld extracted [20] (Appendix A.5).

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 18

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 18

[3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 18

[4] Ronald Azuma and Gary Bishop. Improving static and dynamic registration in an optical see-through HMD. In *Proceedings of SIGGRAPH*, pages 197–204, 1994. 18

[5] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 18

[6] Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations*, 2024. 19

[7] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 18

[8] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems*, 2023. 18

[9] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 18

[10] Yixiang Chen, Peiyan Li, Yuan Xu, Qisen Ma, Jiabing Yang, Kai Wang, Jianhua Yang, Dong An, He Guan, Gaoteng Liu, Jianlou Si, Jun Huang, Jing Liu, Nianfeng Liu, Yan Huang, and Liang Wang. FlowWAM: Optical flow as a unified action representation for world action models. *arXiv preprint arXiv:2607.13017*, 2026. 19

[11] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 8, 9, 18

[12] Yilun Du et al. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems*, 2023. 19

[13] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 6, 8, 12

[14] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 13

[15] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *IEEE International Conference on Robotics and Automation*, 2017. 18

[16] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059), 2025. 18

[17] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 18

[18] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 1, 13

[19] B. Hove and Jean-Jacques E. Slotine. Experiments in robotic catching. In *American Control Conference*, 1991. 18

[20] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2, 19

[21] Wei Huang, Bohan Zhang, Chenzhi Liu, Isabella Liu, Shuai Yang, Weian Mao, Luozhou Wang, Yicheng Xiao, Weifeng Lin, Qixin Hu, Bryan Chu, Sifei Liu, Linxi Fan, Xiaojuan Qi, Song Han, and Yukang Chen. Long-WAM: Scaling the context of world-action models. *arXiv preprint arXiv:2610.10528*, 2026. 19

[22] Seth Hutchinson, Gregory D. Hager, and Peter I. Corke. A tutorial on visual servo control. *IEEE Transactions on Robotics and Automation*, 12(5):651–670, 1996. 18

[23] Haoyuan Ji, Lingxiang Fan, Shang Su, Yinqiao Lu, Mengkai Shi, Jun Gao, and Shuo Feng. DC-WAM: Dynamic-centric visual supervision and reasoning for world-action models. *arXiv preprint arXiv:2607.25918*, 2026. 19

[24] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is better to track together. In *European Conference on Computer Vision*, 2024. 19

[25] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[26] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024. 18

[27] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 18

[28] Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 19

[29] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 9

[30] Xingyu Liu, Charles R. Qi, and Leonidas J. Guibas. FlowNet3D: Learning scene flow in 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019. 19

[31] Yushan Liu, Jingjing Fan, Shoujie Li, Yifan Xie, Xiao-Ping Zhang, and Wenbo Ding. PACT-WAM: Predicting actions and visual foresight with compact temporal encoding for robot manipulation. *arXiv preprint arXiv:2602.15882*, 2026. 19

[32] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 8, 13

[33] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 18

[34] Henrique Morimitsu, Xiaobin Zhu, Roberto M. Cesar, Xiangyang Ji, and Xu-Cheng Yin. DPFlow: Adaptive optical flow estimation with a dual-pyramid framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17810–17820, 2025. 3

[35] E-Ro Nguyen, Yichi Zhang, Kanchana Ranasinghe, Xiang Li, and Michael S. Ryoo. Pixel motion diffusion is what we need for robot control. *arXiv preprint arXiv:2509.22652*, 2025. 19

[36] Octo Model Team, Dibya Ghosh, Homer Walke, et al. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024. 18

[37] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 8, 18

[38] Kanchana Ranasinghe, Xiang Li, E-Ro Nguyen, Cristina Mata, Jongwoo Park, and Michael S. Ryoo. Pixel motion as universal representation for robot control. *arXiv preprint arXiv:2505.07817*, 2025. 19

[39] Kanchana Ranasinghe, Honglu Zhou, Yu Fang, Luyu Yang, Le Xue, Ran Xu, Caiming Xiong, Silvio Savarese, Michael S. Ryoo, and Juan Carlos Niebles. Future optical flow prediction improves robot control & video generation. *arXiv preprint arXiv:2601.10781*, 2026. 19

[40] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 9

[41] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 1, 13, 15

[42] Zachary Teed and Jia Deng. RAFT-3D: Scene flow using rigid-motion embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. 19

[43] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 3

[44] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 19

[45] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546*, 2025. 3

[46] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 8, 19

[47] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 9, 19

[48] Yuxi Xiao, Qianqian Wang, Shangzhan Zhang, Nan Xue, Sida Peng, Yujun Shen, and Xiaowei Zhou. SpatialTracker: Tracking any 2D pixels in 3D space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 19

[49] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 8

[50] Zhenjia Xu, Zhanpeng He, Jiajun Wu, and Shuran Song. Learning 3D dynamic scene representations for robot manipulation. In *Conference on Robot Learning*, 2020. 19

[51] Heng Yu, David D. Yuan, Juze Zhang, Changan Chen, Yao Feng, Michelle Baldonado, Steve Cousins, Li Fei-Fei, Jiajun Wu, and Ehsan Adeli. OpenWAM: An open framework for composable world-action models. *arXiv preprint arXiv:2610.07922*, 2026. 19

[52] Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. Efficiently reconstructing dynamic scenes one D4RT at a time. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 19

[53] Jiahui Zhang, Yurui Chen, Yueming Xu, Ze Huang, Yanpeng Zhou, Yu-Jie Yuan, Xinyue Cai, Guowei Huang, Xingyue

Quan, Hang Xu, and Li Zhang. 4D-VLA: Spatiotemporal vision-language-action pretraining with cross-scene calibration. In *Advances in Neural Information Processing Systems*, 2025. 19

[54] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 8

[55] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 19

[56] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 8, 18

[57] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3D-VLA: A 3D vision-language-action generative world model. In *International Conference on Machine Learning*, 2024. 19

[58] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, 2025. 18

[59] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 3

[60] Yifeng Zhu, Zhenyu Jiang, Peter Stone, and Yuke Zhu. Learning generalizable manipulation policies with object-centric 3D representations. In *Conference on Robot Learning*, 2023. 19

[61] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023. 18